

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing**Practical 01**

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation**Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" branch of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure
Environment
Actuators
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`__init__` method, the variables that define the physical world are `self.width`, `self.height`, `self.agent_pos`, `self.walls`, `self.food_positions`, and `self.opponents`.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent

The code explicitly initializes a `self.opponents` list and moves them randomly throughout the game loop. Because of this, the environment counts as multi-agent since other active entities are operating in the same space and directly impacting the outcome.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is basically the complete, cumulative historical log of every single raw sensory input the agent has ever received, running all the way from the time it was first turned on up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

This represents the Sensors (S). It essentially acts as the mechanism that scoops up raw data from the world and feeds it straight into the agent.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

While the agent gets info on its own position and the opponents' coordinates, the `get_percept()` dictionary only hands back basic boolean flags like `smells_food` or `hit_wall`. That means it can only really "sense" food or walls if it's right next to them or standing directly on top of them. Since those sensory limits keep the agent from seeing the full grid layout all at once, the environment is definitely partially observable.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

This represents the Performance Measure (P). The `self.score` variable acts as the main objective metric used to grade how well the agent is doing.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Measuring internal effort just creates a loophole for the agent to exploit the system. You've got to measure real-world results instead so the machine actually solves the actual problem, rather than just doing pointless busywork purely to pump up its score.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples

safely avoiding position $(0, 0)$, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Hiding this data totally reinforces the environment as partially observable (or even introduces totally unobservable elements). Since the agent has zero sensory access to a major hazard in the world state, it's completely flying blind to that specific feature.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Rationality is all about maximizing expected utility using whatever clues are available in the agent's perceived history. By hooking the agent up with a `smells_toxin` sensor, it finally gets the missing data it needs to actually predict danger. Because of that, it can now make a much smarter choice to back away from the trap, which ultimately maximizes its performance score.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example

discussed in Lecture 01 that illustrated the danger of faulty metrics?

That flawed metric handed the vacuum robot a plus-one point every time it turned on its suction motor to grab some dirt. A rational agent totally exploited this by cleaning a spot, purposely dumping the dirt right back on the floor, and vacuuming the exact same spot over and over forever just to farm infinite points.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING